

Análisis Estático de Código

Hospital Management System

Proyecto Final – Validación y Verificación de Software

Paul Salas (QA / Calidad)
Escuela Politécnica Nacional – 2026A

19 de julio de 2026

Resumen

Este informe documenta la configuración y ejecución de herramientas de análisis estático sobre el backend (Java/Spring Boot) y el frontend (JavaScript vanilla) del proyecto. Todas las herramientas se corrieron realmente contra el código del repositorio; los reportes crudos (`.txt/.xml/.json`) están disponibles en `docs/informes/` para verificación.

Índice

1. Herramientas utilizadas y configuración	1
2. Resumen de hallazgos por severidad	1
3. Checkstyle – desglose	2
4. SpotBugs + FindSecBugs – desglose	2
5. ESLint – desglose	3
6. Análisis de 5+ hallazgos relevantes (requerido por la rúbrica)	4
7. Reportes crudos	4

1. Herramientas utilizadas y configuración

Herramienta	Alcance	Configuración	Invocación
Checkstyle 3.5.0	Backend (Java)	<code>google_checks.xml</code> (Google Java Style), plugin agregado en <code>backend/pom.xml</code>	<code>mvn checkstyle:check</code>
SpotBugs 4.8.6.3 + FindSecBugs 1.13.0	Backend (bytecode de compilado)	<code>effort=Max</code> , <code>threshold=Low</code> , plugin de seguridad FindSecBugs agregado	<code>mvn spotbugs:spotbugs</code>
ESLint 8.57.1	Frontend (<code>js/**/*.js</code> , <code>e2e/**/*.js</code>)	<code>eslint:recommended</code> , <code>.eslintrc.json</code> en <code>frontend/</code> con overrides para tests (jest) y e2e (node)	<code>npx eslint "js/**/*.js" "e2e/**/*.js"</code>

Las tres herramientas quedaron integradas al proyecto (config versionada en el repo), no ejecutadas de forma ad-hoc: cualquier integrante puede reproducir estos resultados con los comandos de la tabla.

2. Resumen de hallazgos por severidad

Herramienta	Blocker/Alto	Critical/Medio	Major/Bajo	Total
Checkstyle	0	0	937	937
SpotBugs + Find-SecBugs	0	9 (Medium)	28 (Low)	37
ESLint	0	0	4 (warning)	4
Total general	0	9	969	978

Ninguna herramienta reportó hallazgos de severidad Blocker/Critical según su propia clasificación. Esto es consistente con lo esperado: los defectos más graves del proyecto (inyección SQL, XSS, falta de autenticación) son de **diseño y lógica de negocio**, no defectos que un analizador estático genérico detecte por patrones de bytecode/AST – ver [analisis-owasp.pdf](#) para esos hallazgos, obtenidos con pruebas dirigidas en vez de SAST genérico.

3. Checkstyle – desglose

Regla	Ocurrencias	Descripción
Indentation	553	El código usa 4 espacios; Google Style exige 2.
RightCurlyAlone / LeftCurly	224	Estilo de llaves distinto al de Google (posición de } / {).
EmptyLineSeparator	56	Faltan/sobran líneas en blanco entre miembros de clase según la convención.
CustomImportOrder	31	Orden de imports no sigue el orden lexicográfico esperado (ej. <code>java.util.List</code> debe ir antes que los paquetes <code>org.springframework.*</code>).
MissingJavadocMethod / Type	47	Clases y métodos públicos sin comentario Javadoc.
LineLength	13	Líneas mayores a 100 caracteres.
AvoidStarImport	8	Imports con <code>*</code> (ej. <code>import jakarta.persistence.*</code> ; en los modelos).
AbbreviationAsWordInName	4	Nombres con abreviaciones en mayúscula (ej. <code>DT0</code>) que Google Style prefiere en <code>PascalCase</code> parcial.

Lectura: el 96 % de las violaciones (777/937) son puramente de formato (indentación, llaves, imports) y se resolverían en un solo paso con un formateador automático (`google-java-format`). El resto (Javadoc faltante) es deuda documental real que sí requiere trabajo manual. Cero violaciones afectan la corrección funcional del código.

4. SpotBugs + FindSecBugs – desglose

Patrón	Ocurrencia	Severidad	Descripción
SPRING_ENDPOINT	28	Low	Informativo: marca cada método público de los 4 <code>@RestController</code> como endpoint expuesto (recordatorio de revisar autorización en cada uno – ver Hallazgo 1 de analisis-owasp.pdf).
EI_EXPOSE_REP2	6	Medium	Constructores/setters de <code>Cita</code> y <code>HistoriaClinica</code> guardan referencias directas a objetos mutables (<code>Doctor</code> , <code>Paciente</code>) recibidos como parámetro, sin copiarlos.
EI_EXPOSE_REP	3	Medium	Los getters de esas mismas clases devuelven la referencia interna mutable directamente, permitiendo que código externo modifique el estado de la entidad sin pasar por sus setters.

Análisis del hallazgo EI_EXPOSE_REP2 (Cita.java:38)

```
public Cita(Long pacienteId, Doctor doctor, LocalDateTime fechaHora,
            String motivo, String estado) {
    this.pacienteId = pacienteId;
    this.doctor = doctor; // <- SpotBugs: EI_EXPOSE_REP2 aqui
    ...
}
```

Si quien llama al constructor conserva su propia referencia a `doctor` y la modifica después, esa modificación se refleja también dentro de la `Cita` ya creada, rompiendo encapsulamiento. En este proyecto el impacto práctico es bajo (Hibernate gestiona el ciclo de vida de las entidades), pero es un patrón a evitar en general.

Nota importante (falso negativo): SpotBugs + FindSecBugs, con `effort=Max` y `threshold=Low` (la configuración más sensible disponible), **no detectó** la inyección SQL real en `DoctorService.buscarPorEspecialidad` (concatenación de strings hacia `entityManager.createNativeQuery`). Esa vulnerabilidad solo se evidenció con pruebas de integración dirigidas – ver Hallazgo 4 de [analisis-owasp.pdf](#). Se documenta explícitamente porque es una lección metodológica central del proyecto: **el análisis estático no reemplaza las pruebas de seguridad dirigidas.**

5. ESLint – desglose

```
frontend/js/citas.js
  11:7 warning 'CitasModule' is assigned a value but never used

frontend/js/doctores.js
  10:7 warning 'DoctoresModule' is assigned a value but never used

frontend/js/historias.js
  10:7 warning 'HistoriasModule' is assigned a value but never used

frontend/js/pacientes.js
  11:7 warning 'PacientesModule' is assigned a value but never used

4 problems (0 errors, 4 warnings)
```

Análisis: los 4 warnings son **falsos positivos** propios de la arquitectura del frontend (sin bundler ni módulos ES): cada objeto `XModule` se usa exclusivamente desde atributos `onclick="..."` dentro de cadenas HTML generadas dinámicamente (ej. `onclick="PacientesModule.editarPaciente({p.id})"`), algo que ESLint no puede rastrear estáticamente porque no es una referencia directa en el AST de JavaScript. No representan código muerto real: eliminar esos objetos rompe la aplicación. Se documentan igual porque son la salida real del linter, y porque evidencian una limitación conocida de este estilo de arquitectura (dificulta el análisis estático) – otro argumento a favor de migrar a módulos ES si el proyecto creciera.

6. Análisis de 5+ hallazgos relevantes (requerido por la rúbrica)

1. **Checkstyle – Indentation (553 casos):** el proyecto completo usa 4 espacios; adoptar Google Style (2 espacios) de forma retroactiva tocaría prácticamente cada archivo. Se recomienda decidir un estándar de equipo *antes* de escribir más código, no después.
2. **Checkstyle – MissingJavadocMethod/Type (47 casos):** ninguna clase pública del backend tiene Javadoc. Priorizar los repositorios con `@Query` personalizado (ej. `PacienteRepository.buscarPorNombre`) donde un comentario explicando el comportamiento real de la query evitaría la confusión encontrada en este mismo proyecto (ver nota sobre el comentario engañoso de esa query en `analisis-owasp.pdf`).
3. **SpotBugs – EI_EXPOSE_REP2 en HistoriaClinica (HistoriaClinica.java:38-39):** expone directamente `Paciente` y `Doctor`, dos entidades que contienen datos personales/sanitarios. Aunque el riesgo de mutación accidental es bajo en este proyecto, es exactamente el tipo de hallazgo que en un sistema real con más consumidores de la API se vuelve relevante.
4. **SpotBugs – falso negativo en la inyección SQL real:** documentado arriba. Es el hallazgo metodológicamente más importante de todo el análisis estático: demuestra que la cobertura de herramientas SAST genéricas no es completa y debe complementarse con pruebas de seguridad explícitas.
5. **ESLint – 0 errores reales, 4 falsos positivos estructurales:** el resultado "limpio" de ESLint **no** significa ausencia de problemas de seguridad en el frontend – los hallazgos XSS reales (múltiples usos de `innerHTML` sin escapar) no son detectables por `eslint:recommended` sin un plugin de seguridad específico (ej. `eslint-plugin-no-unsanitized`), que no estaba configurado. Se documenta como área de mejora para el checklist de calidad del equipo.
6. **Checkstyle – CustomImportOrder (31 casos) en los 4 repositorios:** el orden de imports incorrecto es cósmético, pero su repetición idéntica en `PacienteRepository`, `DoctorRepository`, `CitaRepository` y `HistoriaClinicaRepository` sugiere que se generaron a partir de la misma plantilla – un buen candidato para una regla de `.editorconfig` + formateador automático en pre-commit.

7. Reportes crudos

Disponibles en `docs/informes/` para verificación independiente: `checkstyle-report.txt`, `checkstyle-result.xml`, `spotbugs-report.txt`, `spotbugs-result.xml`, `eslint-report.txt`, `eslint-report.json`.